

Week 6 Write Up for
Master of Science
Information and Communications Technology

Melissa Knapp
University of Denver University College

May 9, 2026

Faculty: Nathan Braun, MA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Table of Contents

Table of Contents	i
Overview	1
Challenges Faced.....	1
JUnit Testing.....	2
Future Considerations.....	5

Overview

The primary challenge pertains to tracking visitors across various parking facilities and invoicing them monthly according to the duration of their parking. The system encompasses functionality for registering customers and their vehicles, as well as documenting parking locations and time entries. This week, we used the Decorator pattern to separate different ways of charging the users of the parking lot so that they can be combined with the fewest classes made.

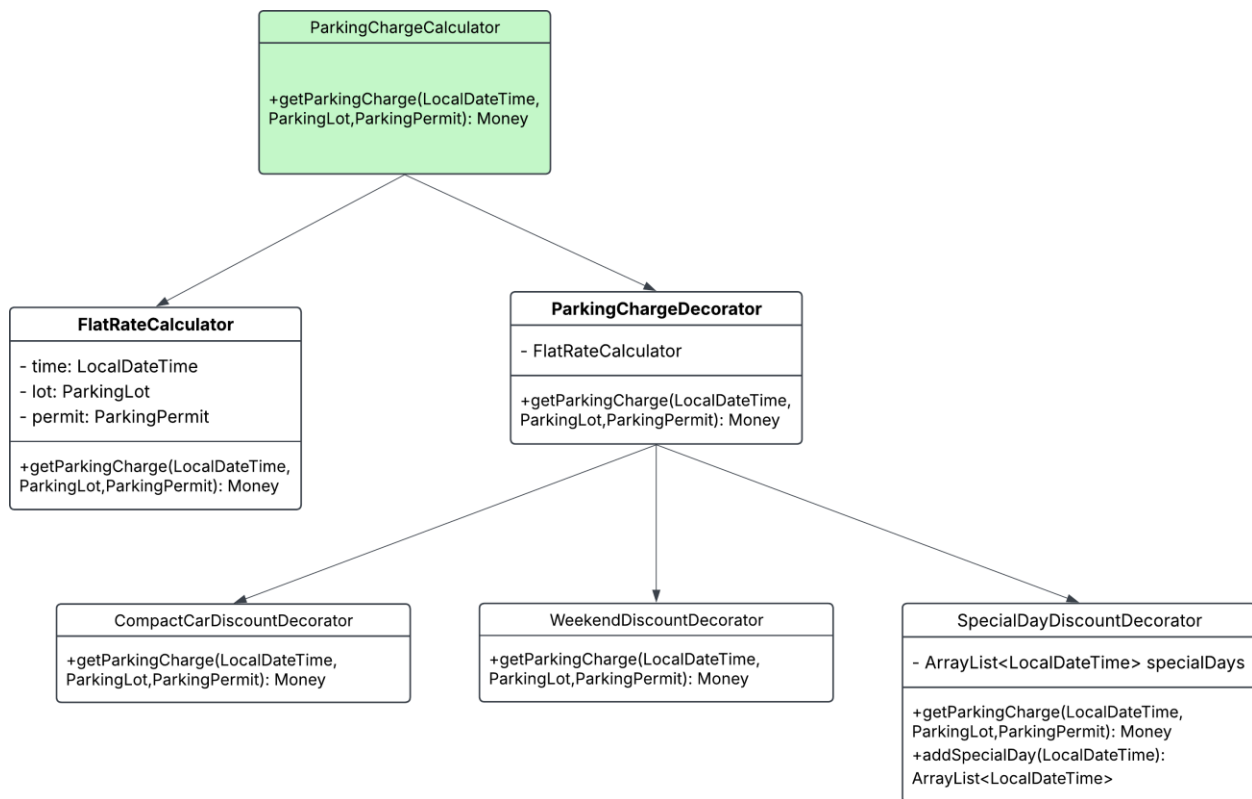


Figure 1. UML Diagram

Challenges Faced

I found this week to be a bit challenging. I found that using the guideposts for the assignment was very helpful because it laid out a skeleton of the basics of the Decorator pattern. I also decided to continue watching the LinkedIn Learning path that the design videos

were pulled from. I thought if I could see someone else set up the Decorator pattern, it would make it a bit easier for me. And it was much simpler after watching the instructor implement the Decorator pattern.

Once I had made the Decorator classes, I wasn't quite sure how to integrate them with the existing codebase. Mainly, I wasn't sure if the strategies made last week were still going to be used in the new codebase, or if the Decorator classes were replacing them. I decided to keep them in and have them invoke the Decorator classes, which might make it a bit more complex than necessary, but I felt it was a good compromise.

One of the other things I've been struggling with in the last few weeks is the calculation of hourly vs daily charging parking lots. From what I understand there are 2 types of lots, those that charge hourly and those that charge daily. So, I decided to add a new attribute to the `ParkingLot`, along with creating an `ENUM` class for the types of charging methods.

JUnit Testing

I also refactored the strategy classes and changed how they did their calculations. But the thing I had a lot of trouble with was calculations with doubles, specifically getting the correct numbers in the JUnit test results. I was trying to get a percentage of weekend days or holidays when compared to the entire parking stay. So, I was using the `.between()` function of `LocalDateTime` to get the distance between entrance and exit. Then I looped through the days between and checked if they matched the condition of the discount strategy. The only problem is that every time I checked it, the result was 0.0. I eventually realized that I was dividing two ints and the result was always less than 1, so it rounded down to 0. Once I figured that out, I cast the ints as doubles and continued the calculations, and they finally worked as expected.

I also went through and reworked all the JUnits that I previously set up to make sure they match the current code base. One of the things I noticed though was that when I ran the tests individually, the ids were different than if I tested the entire codebase at once. So, I decided to change the assertEquals in every id test to assertTrue. Then I just used OR to compare the resulting id to the 2 possible ids, which solved that problem.

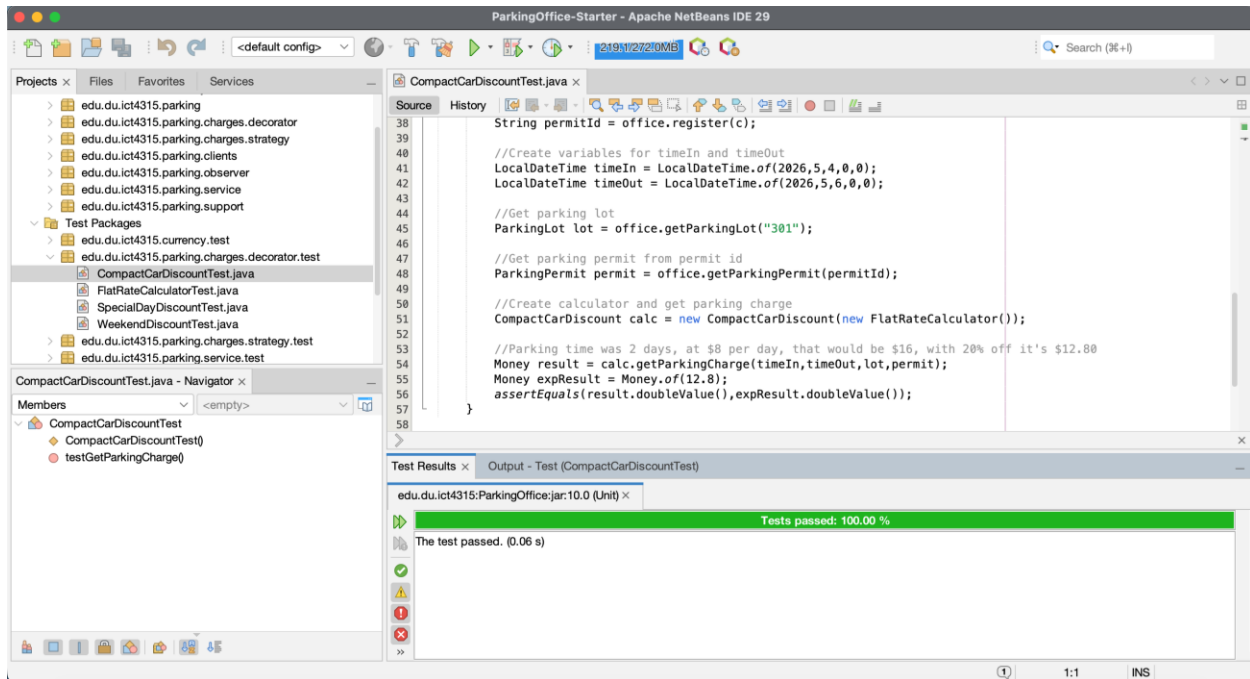


Figure 2. JUnit test results for CompactCarDiscountTest

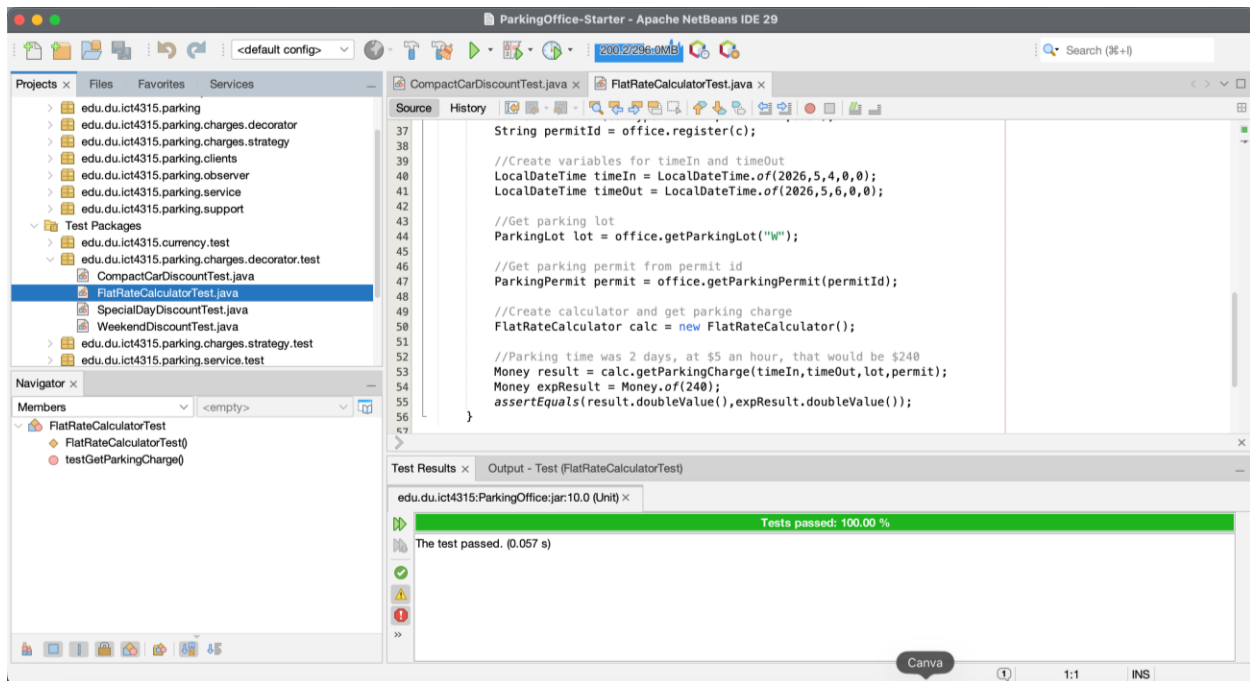


Figure 3. JUnit test results for FlatRateCalculatorTest

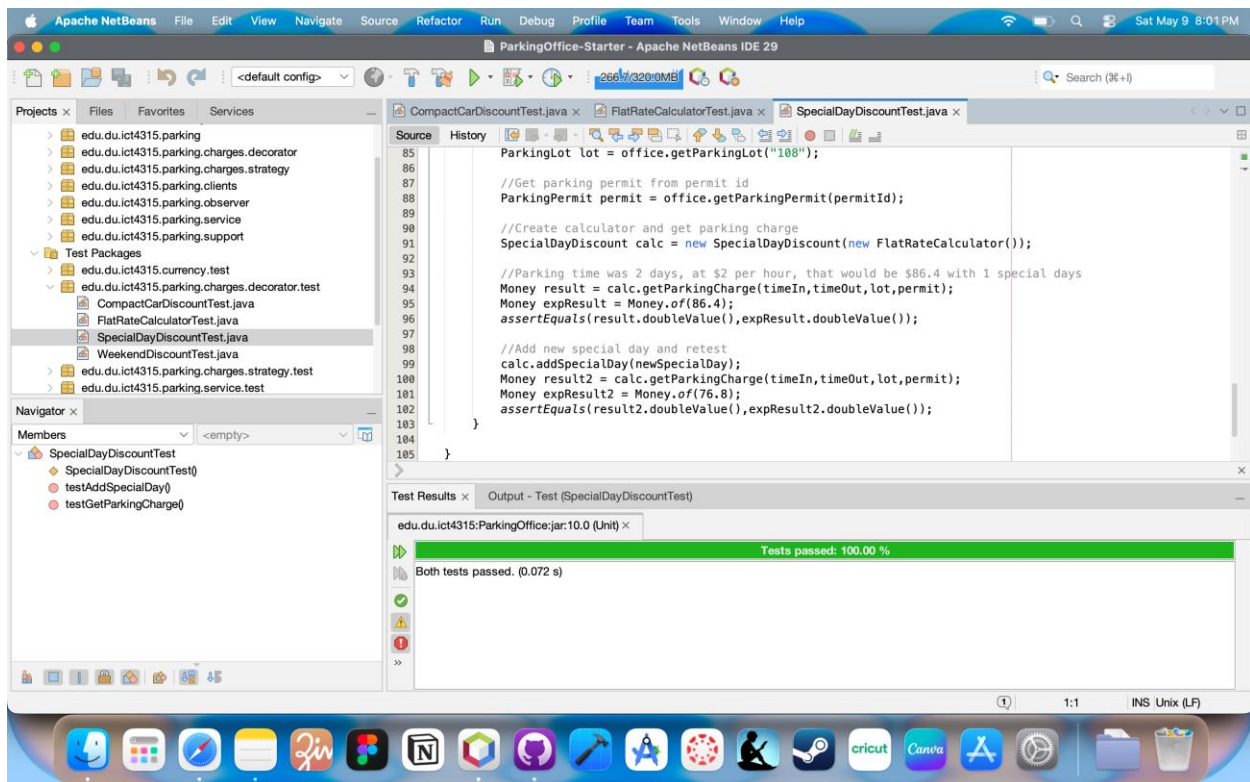


Figure 4. JUnit test results for SpecialDayDiscountTest

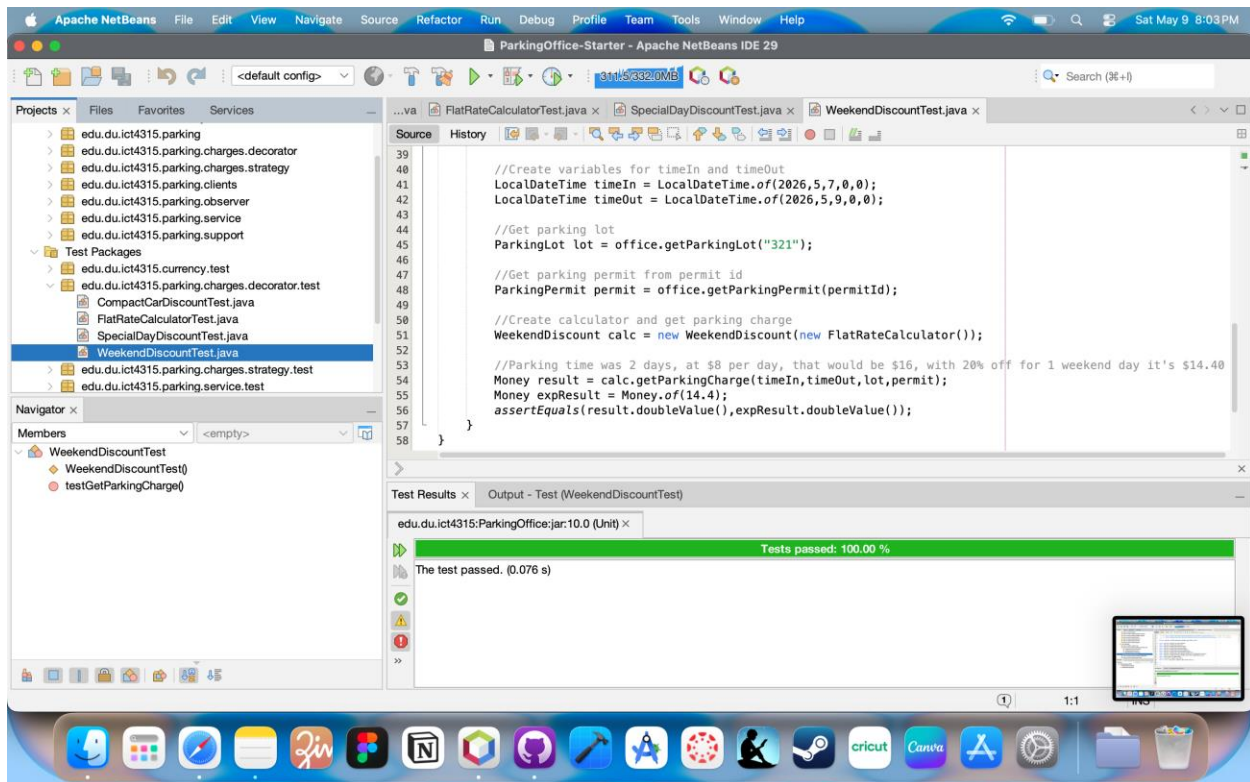


Figure 5. JUnit test results for WeekendDiscountTest

Future Considerations

As I noted in last week's write-up, I want to better understand JUnit's BeforeEach and BeforeAll methods. Since I'm not yet comfortable with them, learning how they work should reduce the repetitive setup of creating objects for each test and help streamline my testing workflow. From what I understand, BeforeEach runs before every test, while BeforeAll runs once before all tests. With a stronger grasp of both, I expect my tests to be more organized and efficient, with less duplicated code and a more consistent test environment.